CS 7180: Agent Architectures & Security

John Alexis Guerra Gomez
jguerra at northeastern.edu
Class: **johnguerra.co/classes/aiCoding_spring_2026**
Slides: **johnguerra.co/lectures/ai_assisted_coding**

What We'll Cover Today

1. Where We Are -- Week 13 checkpoint
2. Agent Fundamentals
3. Anthropic's 6 Agent Patterns
4. Claude Agent SDK
5. Multi-Agent Coordination
6. Agent Safety & Evaluation
7. Security of AI-Generated Code
8. The 8-Gate Security Pipeline
9. Slopsquatting
10. Ethics & Professional Responsibility
11. Hands-on Lab

Where We Are

> *Week 13 -- Agent architectures and securing AI code*

Recap: Week 12

Week 12: Claude Code Extensibility -- skills, hooks, MCP servers, custom sub-agents, parallel sessions
You learned how to **extend** Claude Code. Now you learn how to **build agents from scratch** and how to **secure everything AI generates**.

This Week: Two Sessions

Session 1: Agent Architectures

- What makes something an "agent" vs a scripted workflow
- The 6 canonical patterns from Anthropic research
- Claude Agent SDK for building programmatic agents

Session 2: Security of AI-Generated Code

- The data is alarming -- 45% of AI code has OWASP vulnerabilities
- The 8-gate pipeline that catches what AI misses
- Novel threats like slopsquatting
- Ethics, IP, and professional responsibility

Agent Fundamentals

> *What makes something an "agent"?*

The Augmented LLM

Before agents, understand the building block: the **augmented LLM**.

An LLM augmented with **tools** (APIs, code execution), **retrieval** (search, RAG), and **memory** (conversation history, CLAUDE.md) is the building block for everything that follows.

Workflows vs Agents

This is the most important distinction in this lecture.

Workflows: LLM calls orchestrated by **predefined code paths**

- You (the developer) define the control flow
- The LLM fills in specific steps, but the sequence is fixed
- Predictable, testable, deterministic

Agents: LLM **dynamically determines** its own control flow

- The model decides what to do next based on results
- It selects tools, plans steps, and adapts
- More autonomous, more powerful, less predictable

When to Use What

Dimension	Workflow	Agent
Control	Developer defines steps	Model decides steps
Predictability	High -- same input, same path	Variable -- adapts to results
Complexity	Low-medium	Medium-high
Debugging	Easy -- trace the code path	Harder -- trace the model's reasoning
Best for	Repetitive, well-defined tasks	Open-ended, exploratory tasks
Cost	Lower (fewer LLM calls)	Higher (many LLM calls)

Rule of thumb from Anthropic: Start with the simplest solution. Only add agent complexity when simpler patterns fail.

Anthropic's 6 Agent Patterns

> *From "Building Effective Agents" research*

Pattern 1: Prompt Chaining

Sequential processing with **quality gates** between steps.

How it works:

- Break a task into sequential subtasks
- Each step's output feeds the next step's input
- Quality gates between steps catch errors early

Example: Generate code -> Validate syntax -> Write tests -> Verify tests pass

When to use: Tasks that decompose into fixed, dependent steps.

Pattern 2: Routing

Classify input, then dispatch to a **specialized handler**.

How it works:

- A classifier LLM categorizes the input
- Routes to specialized prompts, tools, or models
- Each handler is optimized for its category

Example: Customer support -- route billing questions to billing agent, technical questions to tech agent.
When to use: Inputs vary widely and benefit from specialization.

Pattern 3: Parallelization

Run independent subtasks **simultaneously** for speed or diversity.

Two sub-patterns:

- **Sectioning:** Split work into independent pieces (e.g., analyze frontend and backend separately)
- **Voting:** Run the same task multiple times for diverse perspectives (e.g., 3 code reviews, majority rules)

When to use: Tasks with independent subtasks or where multiple perspectives improve quality.

Pattern 4: Orchestrator-Workers

A central orchestrator **dynamically delegates** to worker agents.

How it works:

- Orchestrator analyzes the task and creates a plan
- Dynamically spawns worker subtasks (number and type vary)
- Workers execute independently
- Orchestrator synthesizes results

Key difference from parallelization: The orchestrator decides *what* subtasks to create at runtime. The plan is not predefined.

Example: "Refactor the auth system" -- orchestrator identifies 5 files to change, creates a worker for each.

Pattern 5: Evaluator-Optimizer

Generate, then **critique and refine** in a loop.

How it works:

- Generator produces an initial output
- Evaluator scores or critiques the output
- If below threshold, evaluator sends feedback to generator
- Generator revises based on feedback
- Loop continues until quality is sufficient

Example: Write documentation -> Review for completeness -> Revise -> Review again -> Accept.
When to use: Tasks with clear quality criteria where iterative refinement adds value.

Pattern 6: Autonomous Agents

Full **agentic loop** with environment interaction and self-directed planning.

This is Claude Code. The model controls its own loop -- plans, selects tools, executes, observes results, and adapts.
Tradeoff: Most capable but hardest to control. Requires strong guardrails, sandboxing, and human oversight.

Choosing the Right Pattern

Pattern	Complexity	Control	Best For
Prompt Chaining	Low	High	Sequential, well-defined tasks
Routing	Low	High	Varied inputs, specialized handling
Parallelization	Medium	High	Independent subtasks, consensus
Orchestrator-Workers	Medium	Medium	Dynamic decomposition
Evaluator-Optimizer	Medium	Medium	Quality-critical outputs
Autonomous Agents	High	Low	Open-ended, complex tasks

Start simple. Only escalate to more complex patterns when simpler ones are insufficient.

Claude Agent SDK

> *Building programmatic agents in Python and TypeScript*

What Is the Agent SDK?

The Claude Agent SDK lets you **build your own agents** programmatically.

- Available in **Python** and **TypeScript**
- Uses Claude as the reasoning engine
- You define tools, sessions, and hooks
- The SDK handles the agentic loop

SDK vs CLI:

- **CLI** (Claude Code) = interactive development tool for humans
- **SDK** = programmatic building block for embedding agents in your apps

The query() Function

The core of the SDK -- send a task, get an autonomous agent response.

```
from claude_agent_sdk import Agent

agent = Agent(
    model="claude-sonnet-4-20250514",
    tools=[read_file, write_file, run_tests],
    system_prompt="You are a code review agent."
)

result = agent.query(
    "Review the pull request in /tmp/pr-diff.patch "
    "and suggest improvements."
)

print(result.response)
```

The agent autonomously decides which tools to call, in what order, and when to stop.

Built-in Tools

The SDK provides tools out of the box:

```
from claude_agent_sdk.tools import (
    computer,       # Desktop automation
    text_editor,    # File reading and editing
    bash,           # Shell command execution
)

agent = Agent(
    model="claude-sonnet-4-20250514",
    tools=[computer, text_editor, bash],
)
```

You can also define **custom tools** with standard function signatures and docstrings. The SDK converts them to Claude's tool-use format automatically.

Hooks for Control

Hooks let you **intercept** the agentic loop at key points:

```
def pre_tool_use(tool_name, tool_input):
    """Called before every tool invocation."""
    if tool_name == "bash" and "rm -rf" in
tool_input:
        raise PermissionError("Destructive command
blocked")
    return tool_input  # Allow it

def post_tool_use(tool_name, tool_output):
    """Called after every tool invocation."""
    log(f"Tool {tool_name} returned
{len(tool_output)} chars")
    return tool_output

agent = Agent(
    hooks={"pre_tool_use": pre_tool_use,
           "post_tool_use": post_tool_use}
)
```

Hooks give you **guardrails without giving up autonomy**.

Permissions with --allowedTools

Scope what an agent can do:

```
agent = Agent(
    model="claude-sonnet-4-20250514",
    allowed_tools=["text_editor", "bash(npm test)"],
    # Can read/edit files and run tests
    # Cannot run arbitrary bash, make network calls,
etc.
)
```

Principle of least privilege: Give the agent only the tools it needs for its specific task.

Multi-Agent Coordination

> *Agent teams and message passing*

Subagents and Agent Teams

Orchestrator-Worker in code: The orchestrator (Sonnet/Opus) plans and spawns worker agents (Haiku) for subtasks. Workers execute independently and return results.

Writer/Reviewer pattern:

```
Writer Agent ---> Reviewer Agent
     ^
  |
     |     Feedback     |
     +------------------+
       Loop until approved
```

- **Writer** generates code with full tools
- **Reviewer** checks quality, security, tests
- This is the **evaluator-optimizer** applied to code generation

Message Passing Between Agents

Agents communicate through **structured outputs**:

1. Agent 1 (Architect) produces a plan
2. Orchestrating code passes the plan to Agent 2 (Implementer)
3. Agent 2's output passes to Agent 3 (Tester)

Each agent has its own system prompt, tools, and specialization. The orchestrating code passes context between them as strings or structured data.

Model routing: Use powerful models (Opus/Sonnet) for orchestrators and cheaper models (Haiku) for workers to control costs.

Agent Safety & Evaluation

> *Controlling autonomous systems*

The Safety Challenge

Agents are powerful because they are autonomous. But autonomy means:

- **Unintended actions** -- the agent does something you didn't expect
- **Cascading errors** -- one bad tool call leads to more bad calls
- **Data exposure** -- agent accesses or leaks sensitive information
- **Resource consumption** -- agent loops indefinitely, burning tokens/compute

You must design safety in from the start, not bolt it on later.

Sandboxing Strategies

Strategy	Implementation	Protection Level
Container isolation	Docker, VM	Full -- agent cannot affect host system
Tool allowlists	`allowed_tools=["read", "grep"]`	Medium -- limits what agent can do
Hooks	`pre_tool_use` validation	Medium -- block specific actions
Network isolation	No outbound network	High -- prevents data exfiltration
Time/token limits	Max iterations, token budget	Medium -- prevents runaway agents
Human-in-the-loop	Approval for destructive actions	High -- human reviews critical steps

Defense in depth: Combine multiple strategies. No single strategy is sufficient.

Testing Agents Systematically

Agents are harder to test than deterministic code. Four approaches:

1. **Eval suites** -- predefined tasks with known correct outcomes
2. **Trajectory analysis** -- check the agent's reasoning path, not just output
3. **Boundary testing** -- deliberately give ambiguous or adversarial inputs
4. **Cost monitoring** -- track token usage per task to catch runaway agents

The eval mindset from earlier weeks applies here. You're evaluating an agent, not just code.

The Security Problem

> *Session 2: AI-generated code is dangerously insecure*

The Data Is Alarming

Veracode 2025 Study:

Metric	Result
AI code with OWASP Top 10 vulnerabilities	**45%**
Java code failure rate	**72%**
XSS vulnerability rate	**86%**
Log injection vulnerability rate	**88%**

Critical finding: Larger, more capable models do **NOT** generate more secure code. Security performance has not improved even as models get dramatically better at functional correctness.

More Alarming Data

Apiiro Research:

- Privilege escalation flaws: **+322%** in AI-generated code
- Architectural design flaws: **+153%** in AI-generated code

Aikido 2026 Report:

- **1 in 5** organizations reported serious security incidents from AI-generated code

The AI writes code that *works* but is *vulnerable*. It passes tests but fails security audits.

You ship it. You own it.

Why AI Code Is Insecure

1. **Training data includes vulnerable code** -- learned from millions of repos with known vulnerabilities
2. **Functional correctness != security** -- optimizes for "does it work?" not "is it safe?"
3. **Missing context** -- doesn't know your threat model or compliance requirements
4. **Developer overconfidence** -- code that "looks right" and passes tests gets less scrutiny
5. **Speed vs rigor tradeoff** -- faster development tempts developers to skip security reviews

The 8-Gate Security Pipeline

> *Systematic defense for AI-generated code*

Overview: 8 Gates

Every AI-generated code change should pass through **8 security gates** before reaching production.

```
  Code -> [1] -> [2] -> [3] -> [4] -> [5] -> [6] ->
[7] -> [8] -> Production
        Pre-  Deps   SAST   DAST  Contain License
SecAcc  SBOM
        commit
        er
```

No single gate catches everything. Together, they form defense in depth.

Gates 1-2: Secrets & Dependencies

Gate 1 -- Pre-Commit Secrets Detection (Gitleaks)

Scans code before commit for API keys, tokens, passwords, private keys.

```
gitleaks protect --staged   # Run as pre-commit hook
```

AI models sometimes hallucinate credentials or copy patterns that include hardcoded secrets.

Gate 2 -- Dependency Scanning (npm audit, Dependabot, Snyk)

AI suggests dependencies from training data -- some with known vulnerabilities or unmaintained.

```
npm audit          # Check for known vulnerabilities
npm audit fix      # Auto-fix what you can
```

The model may suggest outdated package versions with known CVEs.

Gates 3-4: SAST and DAST

Gate 3 -- SAST (Static Application Security Testing)

Analyzes source code without running it.

- **SonarQube** -- comprehensive, many languages
- **Semgrep** -- lightweight, pattern-based rules

Catches: SQL injection patterns, XSS sinks, hardcoded secrets, insecure crypto

Gate 4 -- DAST (Dynamic Application Security Testing)

Tests the **running application** from the outside.

- **OWASP ZAP** -- open source, automated scanning
- Simulates attacks against your deployed app

Catches: Authentication bypasses, CORS misconfigurations, exposed endpoints

Gates 5-6: Container & License

Gate 5 -- Container Scanning

If you deploy in containers, scan the image:

- Base image vulnerabilities
- Unnecessary packages
- Running as root (don't)

Gate 6 -- License Compliance

Tool: FOSSA, license-checker

AI-generated code may introduce dependencies with **incompatible licenses**.

- GPL dependency in an MIT project? License violation.
- **Red Hat found 17 incidents** of GPL code appearing in MIT-licensed projects via Copilot.

FOSSA scans your dependency tree for license conflicts automatically.

Gates 7-8: Security Criteria & SBOM

Gate 7 -- Security Acceptance Criteria

Add to your Definition of Done: input validation, auth checks, no secrets in code/logs, rate limiting, non-leaking error messages.

Gate 8 -- SBOM (Software Bill of Materials)

A complete inventory of every component. Formats: SPDX, CycloneDX.

Required by U.S. Executive Order 14028, EU Cyber Resilience Act, and enterprise procurement.

```
npx @cyclonedx/cyclonedx-npm --output-file sbom.json
```

Slopsquatting

> *A novel threat unique to AI development*

What Is Slopsquatting?

AI models sometimes **hallucinate package names** that don't exist.

```
You: "How do I parse CSV in Python?"
AI: "Use the fast-csv-parser package: pip install
fast-csv-parser"
```

Problem: `fast-csv-parser` doesn't exist. But what if an attacker **registers it**?

Slopsquatting: Attackers monitor AI hallucinations, register the fake package names, and fill them with malicious code.

When developers follow AI advice and `pip install` or `npm install` the hallucinated package, they install malware.

Defending Against Slopsquatting

1. **Verify every package** -- check it exists on npm/PyPI with real downloads
2. **Use lockfiles** -- `package-lock.json` pins known-good versions
3. **Dependency scanning** (Gate 2) catches known malicious packages
4. **Be suspicious of unfamiliar packages** -- if you've never heard of it, verify first

This is not theoretical -- it's happening in production.

Ethics & Professional Responsibility

> *You ship it, you own it*

IP and Copyright

U.S. Copyright Office (2023):

> *Wholly AI-generated content is **not copyrightable**.*

Your code is only protected if there is meaningful human creative contribution.

Doe v. GitHub (ongoing):

- Potential **$9 billion** in DMCA damages
- Alleges Copilot reproduces copyrighted code without attribution
- Class action on behalf of open-source developers

Red Hat (2024):

- Documented **17 incidents** of GPL-licensed code appearing in MIT-licensed projects via Copilot
- GPL requires derivative works to also be GPL -- violation means legal liability

Professional Responsibility

ACM Code of Ethics, Principle 1.6:

> *"Accept full responsibility for their own work."*

When you use AI to generate code:

- **You** are the author of record
- **You** are responsible for bugs, vulnerabilities, and license violations
- **You** must review every line before shipping
- "The AI wrote it" is not a defense

The standard has not changed. AI is a tool. The engineer is accountable.

Bias in AI-Generated Code

AI perpetuates biases from training data:

- **Default assumptions** -- user schemas assuming binary gender, Western names, English-only
- **Exclusionary patterns** -- accessibility features omitted unless explicitly requested
- **Cultural bias** -- dates, currencies, addresses default to U.S. conventions

Your responsibility: Review for inclusivity, explicitly prompt for accessibility, test with diverse personas, don't ship defaults uncritically.

Hands-on Lab

> *Build a simple agent and audit AI code for security*

Exercise 1: Build an Agent (30 min)

Choose **one** of the 6 patterns and implement it with the Claude Agent SDK:

Option A -- Prompt Chaining:

Build an agent that (1) reads a function, (2) generates a docstring, (3) validates the docstring matches the function signature.

Option B -- Evaluator-Optimizer:

Build an agent that (1) writes a unit test, (2) evaluates if the test is meaningful (not trivially passing), (3) revises until the evaluator approves.

Option C -- Routing:

Build an agent that classifies input as "bug report," "feature request," or "question" and routes to specialized handlers.

Exercise 2: Security Audit (20 min)

Take a piece of AI-generated code from your P3 project and run it through as many gates as you can:

```
# Gate 1: Secrets
gitleaks detect --source .

# Gate 2: Dependencies
npm audit

# Gate 3: SAST (if you have Semgrep)
semgrep --config auto src/

# Check: any vulnerabilities found?
```

Document what you find. Discuss with your team:

- Were there any surprises?
- Would you have caught these without the tools?

This Week's Deliverables

Due This Week

Weekly Quiz 13 -- Agent Architectures & Security concepts
P3 Sprint 3 -- continue your team project
Recommended:

- Try at least one Agent SDK pattern from the lab
- Run `npm audit` and `gitleaks detect` on your P3 repo
- Review your P3 code for the OWASP Top 10

Next Week Preview

Week 14: Emerging AI Engineering

- AI code review automation (Claude Code in GitHub Actions)
- Prompt caching & cost optimization
- RAG patterns for codebases
- Model routing & selection (Opus vs Sonnet vs Haiku)
- Monitoring, observability & security (OpenTelemetry, OWASP)
- Future of AI engineering
- **Demo preparation workshop** for P3

P3 Sprint 4 -- deploy & polish.

Resources

Required Reading

Resource	URL
Building Effective Agents	**anthropic.com/engineering/building-effective-agents**
Claude Agent SDK Overview	**platform.claude.com/docs/en/agent-sdk/overview**
Claude Agent SDK Quick Start	**platform.claude.com/docs/en/agent-sdk/quickstart**
Tool Use (Function Calling)	**platform.claude.com/docs/en/build-with-claude/tool-use/overview**
OWASP Top 10	**owasp.org/www-project-top-ten**

Recommended Reading

Resource	URL
Claude Agent SDK Demos	**github.com/anthropics/claude-agent-sdk-demos**
Prompt Chaining Guide	**docs.anthropic.com/en/docs/build-with-claude/prompt-engineering/chain-prompt**
OpenSSF AI Code Security Guide	**best.openssf.org/Security-Focused-Guide AI-Code-Assistant-Instructions**
Veracode AI Security Analysis	**veracode.com/blog/ai-generated-code-se risks**
Georgetown CSET: Cybersecurity Risks of AI Code	**cset.georgetown.edu/publication/cyberse risks-of-ai-generated-code**

Reference Reading

Resource	URL
Mapping the Mind of a Large Language Model	**anthropic.com/research/mapping-mind-language-model**
Tracing the Thoughts of a Large Language Model	**anthropic.com/research/tracing-thoughts-language-model**
Constitutional AI	**anthropic.com/research/constitutional-ai-harmlessness-from-ai-feedback**

Speaker notes